Why Proxy Wins Don't Equal Deployment Value: Evaluating Policy-Update Procedures End to End

Research Blog

Every machine learning and decision-systems team eventually faces the same release question: *If we let our automated pipeline propose policy updates, screen them on an offline proxy, and ship the winner whenever it clears a threshold, does the deployed system actually improve over keeping the incumbent?*

In practice, teams routinely answer a different question. They inspect the proxy score of the selected candidate—or run a narrow post-hoc check on a single winning policy—and treat that number as the expected value of the update workflow.

That shortcut confuses the score of a selected candidate with the utility of the procedure that produced it. When an engineering organization puts a policy-update workflow into production, it is not deploying a single static policy in a vacuum. It is deploying a composite mechanism: candidate generation, offline screening, an adoption gate that either promotes a candidate or retains the incumbent, and execution in an interactive environment whose scale and feedback dynamics may differ from the screening proxy.

A new paper, *Evaluating the Deployment Utility of a Frozen Policy-Update Procedure: Scale, Completion Preference, and Joint Evidence*, isolates this distinction in a controlled applied study. Rather than proposing a new trading strategy or a universal agent benchmark, the study formulates and executes a bounded policy-evaluation protocol around a single estimand: the **expected incumbent-relative deployment utility of a frozen update-selection procedure**.

The findings offer a sharp cautionary lesson for ML engineering leaders, research managers, and risk owners. Even when an update procedure regularly finds candidates that look positive on a replay proxy, its expected utility in interactive deployment can remain statistically unresolved at small operational scales—and turn decisively negative when deployed at larger scale.

The Deployment Question: Candidate Scores vs. Procedure Utility

Consider how modern ML and quantitative engineering teams iterate on decision policies. Whether tuning execution algorithms, routing logic, or automated bidding rules, the workflow usually follows a standard template:

1. **Generate** a batch of candidate policy modifications from an existing baseline.
2. **Screen** those candidates against logged or replayed historical trajectories using a fast proxy metric.
3. **Select** the top-performing candidate if its proxy improvement exceeds a safety margin; otherwise, **retain the incumbent** (a no-update outcome with zero relative gain).
4. **Deploy** the resulting choice into the live, interactive environment.

When a candidate clears the bar with a +1.3 basis-point (bps) gain on the screening proxy, it is tempting to log +1.3 bps as the expected contribution of the update cycle. Yet three distinct forces drive a wedge between that proxy readout and actual deployment utility:

- **Selection optimism (winner's curse):** Choosing the best of K noisy candidates

systematically inflates the apparent quality of the winner, even if the proxy is unbiased for any single fixed policy.

- **Finite proxy-sample noise:** Screening on finite replay windows introduces estimation error into both candidate ranking and threshold gating.
- **Replay-to-interactive divergence:** A non-reactive replay log cannot capture how an environment responds when a modified policy changes order sizes, timing, or inventory trajectories.

Because of these compounding effects, evaluating the *procedure* requires treating the entire generate-screen-gate-deploy loop as a single frozen object of study. If the procedure decides not to update on a given run, its incumbent-relative utility is zero. If it adopts a candidate, its utility is whatever that candidate actually achieves in interactive execution relative to the incumbent, evaluated under the operator's true completion and cost preferences.

An Illustrative Case: Updating Execution Policies in a Synthetic Market

To make this concrete without confounding from non-stationary live markets, the study examines a frozen replay-proxy policy-update procedure operating in a controlled synthetic continuous double auction (CDA).

In this environment, an execution policy must liquidate or acquire a target inventory over a fixed horizon inside an order book populated by background market participants. The engineering team has an incumbent policy and a frozen update procedure that generates candidate parameter updates, evaluates them on replay data, and either adopts the best candidate or retains the incumbent.

What happens when that selected policy is placed into the interactive market?

At tiny order sizes, the policy's orders barely disturb the book; replay assumptions roughly hold, though run-to-run market noise is high relative to the tiny signal. As the order scale increases, however, the policy's own limit and market orders consume available liquidity, shift queue priority, and leave larger unexecuted remainders at the end of the horizon. If the operator cares deeply about completing the full order—penalizing incomplete fills heavily—a candidate policy that looked thrifty on passive replay data can suffer severe end-of-horizon completion penalties in interactive trading.

Crucially, this failure mode is invisible if you only examine the replay proxy score, and it cannot be diagnosed merely by counting how often a proxy-selected candidate beats the incumbent.

Methodology: Defining the Estimand and Unit of Analysis

The paper's primary contribution is methodological clarity: aligning the statistical protocol with the actual operational unit of a policy-update pipeline.

The Estimand and Experimental Structure

The target estimand is the expected incumbent-relative utility of the frozen update-selection procedure across a specified grid of deployment conditions. In the main synthetic-market study, the experimental design is structured around:

- **50 independent generation roots:** Each root represents an independent run of the candidate-generation process.

- **16 candidates per root:** Within each root, the procedure generates 16 candidate updates for screening.
- **100 root–regime selection decisions:** Each of the 50 independent roots produces one selection decision in each liquidity regime. The independent sample size is 50 roots, not 100 decisions.
- **2,400 paired interactive deployment records:** Selected policies and incumbents are evaluated across paired interactive market rollouts spanning a 12-cell deployment grid.

That 12-cell grid crosses three operational dimensions:

1. **Liquidity regime:** `liquid` vs. `thin` order books.
2. **Order scale (s):** small ($s = 0.002$), medium ($s = 0.05$), and large ($s = 0.3$).
3. **Inventory-response setting (ρ):** $\rho = 0$ vs. $\rho = 1$.

Together with key contrasts across regimes and responses, the original confirmatory design prespecifies a **24-estimate family** evaluated under an original utility definition U_{100}, which applies a **100-basis-point penalty** to incomplete fills at the end of the trading horizon.

Why the Root Is the Independent Unit

A common pitfall in ML systems evaluation is treating every evaluation rollout—here, all 2,400 paired deployment records—as independent observations. Doing so dramatically understates uncertainty.

When a single generation root produces a candidate pool and a selection decision, all interactive rollouts downstream of that root share the same candidate draw and screening noise. The independent experimental unit is therefore the **root** ($n = 50$), not the individual rollout. All valid confidence bands and simultaneous intervals in the study respect this root-level clustering.

Three Elementary Formal Characterizations

To guide how teams interpret deployment data, the paper lays out three elementary mathematical characterizations. These are not deep new theorems or novel statistical estimators; rather, they are straightforward accounting identities that prevent common analytical mistakes:

1. **Reversal frequency cannot determine mean utility:** Teams often track "sign reversals"—how frequently a candidate with a positive proxy score delivers negative utility in deployment. Elementary expectation algebra shows that reversal frequency alone does not determine whether the procedure has positive or negative expected utility. A procedure with a low reversal rate can still be net-negative if its occasional failures are severe (tail losses from incomplete fills), and a high-reversal procedure can be net-positive if its wins outweigh small losses. Gain and loss magnitudes must be retained.
2. **Linear preference frontier under fixed selections:** When deployment utility trades off execution price against an incomplete-fill penalty p (keeping the procedure's adoption choices fixed), expected utility is an affine function of the penalty parameter: $\mu(p) = a + pb$, where a captures the base execution component and b captures the expected completion shortfall relative to the incumbent. This yields a simple linear preference frontier and a closed-form diagnostic break-even penalty without re-running simulations.
3. **Two-level variance decomposition:** Total uncertainty in estimated deployment utility decomposes cleanly into *between-root variation* (differences across candidate genera-

tion and selection draws) and *within-root deployment noise* (interactive rollout variance for a given selection). Knowing which component dominates tells engineering teams whether they need more independent generation roots or more interactive rollouts per root.

Where This Fits in the Literature

To keep its scope sharp, the paper explicitly separates procedure-level deployment evaluation from several adjacent research areas:

- **Off-Policy Evaluation (OPE) and High-Confidence OPE:** Standard OPE methods estimate the value of a *fixed* target policy using logged data from a behavior policy. They do not evaluate an end-to-end procedure that generates random candidate pools, screens them, and deploys them across shifting interactive scales.
- **Safe Policy Improvement (e.g., SPIBB):** Safe improvement algorithms construct policies guaranteed to beat a baseline under specific support assumptions. They are *methods for building policies*, whereas this study audits the empirical deployment utility of an already-frozen update workflow.
- **Conservative evaluation of offline policy learning:** A close journal precedent studies conservative evaluation of offline learning pipelines. The present work shares this commitment to rigorous evaluation of learning workflows, extending the focus to interactive scale sensitivity, completion-penalty frontiers, and joint simultaneous inference across deployment conditions.
- **Active Offline Policy Selection:** Active selection methods adaptively allocate a limited budget of online evaluations to pick the best candidate from a set. Here, the selection rule is already frozen and offline; the question is what happens *after* selection when the procedure's output meets an interactive environment.
- **Agent Reliability and Benchmark Stability (e.g., HAL):** Recent benchmark audits examine ranking stability, scaffold sensitivity, and evaluation variance across generalist AI agents. While complementary in spirit, this study is a bounded applied policy-evaluation analysis, not a general-purpose agent leaderboard.
- **Transition-Level Fidelity (PIVOT and PIVOT-KG):** Prior work on PIVOT investigates transition-level *Improvement Fidelity* and knowledge-guided mechanisms (PIVOT-KG). The present paper evaluates frozen procedure-level deployment utility on its own standalone protocol and neither replaces those methods nor reuses PIVOT experimental results.

The Evidence: What Happened When the Frozen Procedure Met the Market

When the frozen update procedure is evaluated across the prespecified 24-estimate family under the original U_{100} utility metric (100 bps incomplete-fill penalty) using an approximate simultaneous confidence band, a striking pattern emerges across order scales.

1. Largest Scale ($s = 0.3$): Decisively Negative Utility

At the largest order scale ($s = 0.3$), the procedure's expected incumbent-relative utility is unambiguously negative across every liquidity regime and inventory-response setting. None of the simultaneous intervals cross zero:

Regime	Inventory Response (ρ)	Mean Utility U_{100} (bps) [Simultaneous Interval]
liquid	0	-11.9800 [-18.3944, -5.5656]
liquid	1	-12.2848 [-18.6992, -5.8704]
thin	0	-22.0073 [-28.4217, -15.5929]
thin	1	-21.6446 [-28.0590, -15.2302]

In liquid books at $s = 0.3$, deploying the frozen update procedure loses roughly 12.0 to 12.3 bps relative to simply keeping the incumbent policy. In thin books, the degradation nearly doubles, losing 21.6 to 22.0 bps.

The same saved point estimates are interpreted more cautiously in the wider 64-estimate diagnostic family: at 100 bps, the two liquid-regime largest-scale bands include zero, while the thin-regime estimates remain negative. The four negative largest-scale conclusions above belong to the prespecified 24-estimate confirmatory family; the wider 64-estimate result is a post-outcome sensitivity analysis.

2. Lower Scales ($s = 0.002$ and $s = 0.05$) and Contrasts: Unresolved

What about smaller order scales, or differences between inventory-response settings ($\rho = 0$ vs. $\rho = 1$) and interaction contrasts? Under the prespecified 24-estimate family, **all lower-scale cells and response/interaction contrasts remain statistically unresolved**. Their simultaneous intervals straddle zero.

This split outcome—decisively negative at large scale, unresolved at small and medium scales—is a central empirical takeaway. A team testing only at small scales might conclude the update procedure is harmless or mildly promising; a team extrapolating proxy wins to large orders would suffer immediate, statistically significant losses.

3. Post-Outcome Diagnostic: The Proxy-vs.-Deployment Gap at $s = 0.05$

Why did the procedure adopt updates if they did not reliably improve interactive performance? To inspect the mechanism, the authors conduct a separate outcome-aware, post-outcome diagnostic outside the prespecified 24-estimate family at the intermediate scale $s = 0.05$, comparing the saved replay-proxy gain (where a no-update decision is scored as 0) against the paired interactive deployment gain.

Evaluated with a separate whole-root simultaneous interval (outside the original 24-estimate family and without running new simulations), the gap between saved proxy gain and interactive deployment utility at $s = 0.05$ is consistently positive across all four cells:

- **liquid, $\rho = 0$:** 1.314 bps $[0.649, 1.979]$
- **liquid, $\rho = 1$:** 1.294 bps $[0.629, 1.959]$
- **thin, $\rho = 0$:** 0.952 bps $[0.288, 1.617]$
- **thin, $\rho = 1$:** 1.057 bps $[0.392, 1.722]$

In every cell, the replay proxy overstates interactive deployment performance by roughly 1.0 to 1.3 bps.

It is vital to state what this diagnostic is—and what it is not. It is **not** an isolate of causal proxy bias. Because it compares the selected candidate's saved proxy gain (with no-update $= 0$) to its interactive realization, it jointly reflects three entangled factors: winner-selection

optimism across the 16 candidates, finite proxy-sample noise, and structural differences between non-reactive replay and interactive order-book execution.

4. Post-Outcome Sensitivity: Completion Preferences and Break-Even Frontiers

How much of the large-scale failure is driven by the operator's 100-bps penalty on incomplete fills?

To answer this without pretending to run a new confirmatory experiment, the study performs a post-outcome utility sensitivity analysis. Keeping the exact same candidate selections and saved trajectories fixed—**without reselecting candidates** under new objectives—the analysis revalues the outcomes at incomplete-fill penalties of **25, 50, 100, and 200 bps**.

Using the affine preference frontier $\mu(p) = a + pb$ alongside a separate joint component construction, the authors map how expected utility shifts as completion preference varies and compute diagnostic break-even penalties. The four estimated zero crossings range from 0.454 to 5.236 bps; these are post-outcome ratio estimates with a separate uncertainty construction, not a confirmatory threshold. Along with an expanded **64-estimate family**, root-deletion stability checks, and alternative confidence band constructions, these sensitivity tools are transparently labeled as **post-outcome diagnostics**. They illuminate *why* the procedure loses money at scale—and how sensitive conclusions are to operational preferences—without blurring the line between prespecified confirmatory claims and exploratory reanalysis.

Scope Conditions and Explicit Limits

Strong engineering research is defined as much by what it refuses to claim as by what it proves. Decision-makers reading this study should keep five explicit boundaries in mind:

1. **One frozen procedure, one synthetic market:** All primary results apply to a single frozen replay-proxy update procedure inside one controlled synthetic continuous double auction.
2. **No universal scaling laws or real-market execution guarantees:** The degradation at $s = 0.3$ is not a universal mathematical law of policy scaling, nor does it validate real-exchange market microstructure execution or isolate causal proxy bias.
3. **No operational systems overhead:** The utility metric captures execution and completion preferences within the market task; it does not measure API costs, token/compute budgets, inference latency, safety guardrails, or long-horizon multi-day drift.
4. **Auxiliary scope of the planner-code case:** An accompanying planner-code case included in the broader project has only **three generation roots** and is strictly auxiliary; it does not support population-level statistical inference.
5. **Saved-outcome reanalysis:** The sensitivity, break-even, and proxy-gap analyses are reanalyses of saved experimental outcomes, not full raw-model reruns under redesigned selection objectives.

Practical Implications: A Checklist for Deployment Reports

For ML engineering leads and research managers, the operational lesson is clear: **stop approving policy-update pipelines based on candidate-level proxy metrics alone.** When

an automated workflow generates, screens, and gates policy updates, the workflow itself must be audited as a statistical procedure across the conditions where it will actually run.

To translate this principle into engineering governance, teams should require a structured **Deployment Procedure Report** before promoting an automated update workflow. Rather than treating such a report as an automatic green light to deploy, leaders should use it as an auditable specification—and a blueprint for future evaluation tooling.

Every deployment report for a policy-update procedure should explicitly document the following seven elements:

1. **The Frozen Procedure Specification:** Define the exact candidate-generation mechanism, candidate pool size (K), screening dataset or replay window, and hyperparameter configurations locked prior to evaluation.
2. **The Adoption and No-Update Rule:** Specify the exact gating threshold or selection criterion under which a candidate replaces the incumbent, and confirm that no-update outcomes are scored as zero incumbent-relative gain in procedure-level averages.
3. **The Independent Population Unit:** Identify the true independent unit of randomness (e.g., the generation root or episode seed cluster, such as the 50 roots here) rather than inflating sample size by counting correlated downstream rollouts ($2,400$ records) as independent.
4. **The Target Deployment Conditions:** Enumerate the full grid of operational environments (e.g., liquidity regimes, traffic volumes, order scales s, and environmental feedback settings ρ) across which the procedure is expected to operate.
5. **The Utility Function and Operational Costs:** State the exact evaluation objective—including completion penalties (such as the 100-bps penalty in U_{100}) and any unmodeled compute, token, or latency budgets—and note whether sensitivity checks revalue fixed selections ($\mu(p) = a + pb$) or require full reselection.
6. **The Inferential Family and Multiplicity Control:** Clearly separate the prespecified confirmatory family (e.g., the 24-estimate family with simultaneous confidence bands) from post-outcome diagnostics (e.g., expanded 64-estimate grids, proxy-vs.-deployment gap decompositions, or break-even frontiers).
7. **Triaged Conclusions (Positive, Negative, Unresolved):** Report results transparently across the deployment grid by classifying every cell and contrast as *statistically positive*, *statistically negative* (like the $s = 0.3$ cells), or *unresolved* (like the lower-scale cells and response contrasts), resisting the urge to round unresolved intervals up to success.

By separating candidate proxy scores from procedure-level deployment utility—and enforcing strict boundaries between confirmatory inference and post-outcome diagnostics—engineering teams can catch scale-induced failures in the lab before they become expensive surprises in production.